ОНТОЛОГИЯ ПРЕДМЕТНОЙ ОБЛАСТИ ВИДЕОКАРТА

Руководство по открытию и проверке онтологии

Листов 19

Руководитель команды	Шегай Павел Игоревич
Технический писатель	Дергалов Никита Олегович
Тестировщик	Каракешишян Дмитрий Мгерович
Разработчик	Вохрамеев Михаил Викторович
Системный аналитик	Карузин Иван Михайлович

Москва 2026

СОДЕРЖАНИЕ

1 НАЗНАЧЕНИЕ 3

2 ТРЕБОВАНИЯ К СРЕДЕ 5

3 ОТКРЫТИЕ ОНТОЛОГИИ 7

4 ЗАПУСК REASONER 9

5 ПРОВЕРКА ПОЛНОТЫ ЗАПОЛНЕНИЯ 11

6 ВЫПОЛНЕНИЕ КОНТРОЛЬНЫХ ЗАПРОСОВ 12

7 ПРОВЕРКА ОБНАРУЖЕНИЯ ОШИБОК 14

7.1 Подготовка рабочей копии 14

7.2 Тестовые изменения 14

8 СОХРАНЕНИЕ И ПОВТОРНОЕ ОТКРЫТИЕ 16

9 ВОЗМОЖНЫЕ ЗАТРУДНЕНИЯ 17

10 АВТОМАТИЧЕСКИЕ ИСПЫТАНИЯ 18

ПРИЛОЖЕНИЕ А 19

1 НАЗНАЧЕНИЕ

Руководство описывает работу с онтологией предметной области «Видеокарта» (файл videocard.owl, версия 1.0.0) в редакторе Protégé Desktop: открытие файла, запуск reasoner, просмотр выведенной классификации, выполнение восьми контрольных запросов и воспроизведение проверок. Руководство входит в комплект документации по разделу 5 технического задания «Онтология предметной области Видеокарта», версия 1.4 (далее — ТЗ), перечисление 4.

Руководство рассчитано на участников команды и преподавателя. Пользователь должен знать основные операции Protégé и различать классы, свойства, индивиды и аксиомы (п. 3.2 ТЗ). Соответствие разделов руководства испытаниям ТЗ приведено в приложении А.

Состав комплекта приведён в таблице 1 (п. 4.6.4 ТЗ).

Таблица 1 — Состав комплекта

Каталог и файл	Назначение
ontology/videocard.owl	Финальная онтология, RDF/XML, UTF-8
docs/tz_videokarta_ontology_v1.4.docx	Техническое задание с UML-диаграммой и IDEF5-схемой
docs/opisanie_predmetnoy_oblasti_videocard_v1.0.docx	Описание предметной области и словарь понятий
docs/opisanie_ontologii_videocard_v1.0.docx	Описание онтологии, учебный набор, контрольные вопросы с эталонами
docs/rukovodstvo_videocard_v1.0.docx	Настоящее руководство
docs/diagrams/	Редактируемые исходники UML- и IDEF5-диаграмм
docs/ontology_description.md, docs/test_protocol.md	Описание онтологии и протокол автоматического прогона от разработчика (нужны автотесту T11)
ontology/src/	Генератор онтологии и тестовых копий (Python 3, только стандартная библиотека)
tests/negative/	Тестовые копии T06–T09 с намеренными дефектами; не финальная онтология
tests/protokol_ispytaniy_videocard.docx	Протокол испытаний T01–T11 со снимками экрана
tests/run_tests.sh, tests/harness/, tests/expected.json	Автоматические испытания T01–T11 без Protégé и эталонные ответы
tests/results/	Журналы последнего автоматического прогона (T01.log … T11.log, summary.json)
docs/otchet_lr2_videocard_v1.0.docx	Отчёт о выполнении лабораторной работы
README.md	Порядок запуска и обозначение версии

Тестовые копии с намеренными противоречиями хранятся отдельно, в каталоге `tests/negative/`, и не заменяют финальную онтологию (п. 4.5.2 ТЗ). Их можно открыть вместо ручного внесения изменений (раздел 7).

2 ТРЕБОВАНИЯ К СРЕДЕ

Контрольная среда зафиксирована протоколом испытаний (таблица 2). Допускается более новая стабильная версия Protégé с совместимым reasoner (п. 4.6.1 ТЗ).

Таблица 2 — Программные средства

Средство	Версия	Назначение	Обязательность
Protégé Desktop	5.6.9	Открытие и редактирование онтологии	обязательно
HermiT Reasoner	1.4.3.456	Логический вывод, проверка согласованности	обязательно
DL Query	4.0.1	Выполнение контрольных запросов	обязательно
Explanation Workbench	3.0.1	Объяснение выводов и противоречий (кнопка «?», Explain)	рекомендуется
ELK Reasoner	0.6.0	Не используется для приёмки: не поддерживает весь OWL 2 DL	не требуется

Перечисленные модули входят в стандартную сборку Protégé 5.6.9. Сборка для Windows поставляется со встроенной Java, отдельная установка Java не нужна. Испытания проводились в Windows (x64).

Protégé загружается со страницы выпусков проекта на GitHub: github.com/protegeproject/protege-distribution/releases, файл Protege-5.6.9-win.zip (для macOS — Protege-5.6.9-mac.zip, для Linux — Protege-5.6.9-linux.tar.gz).

Версии Protégé и модулей проверяются командой Help → About (рисунок 1).

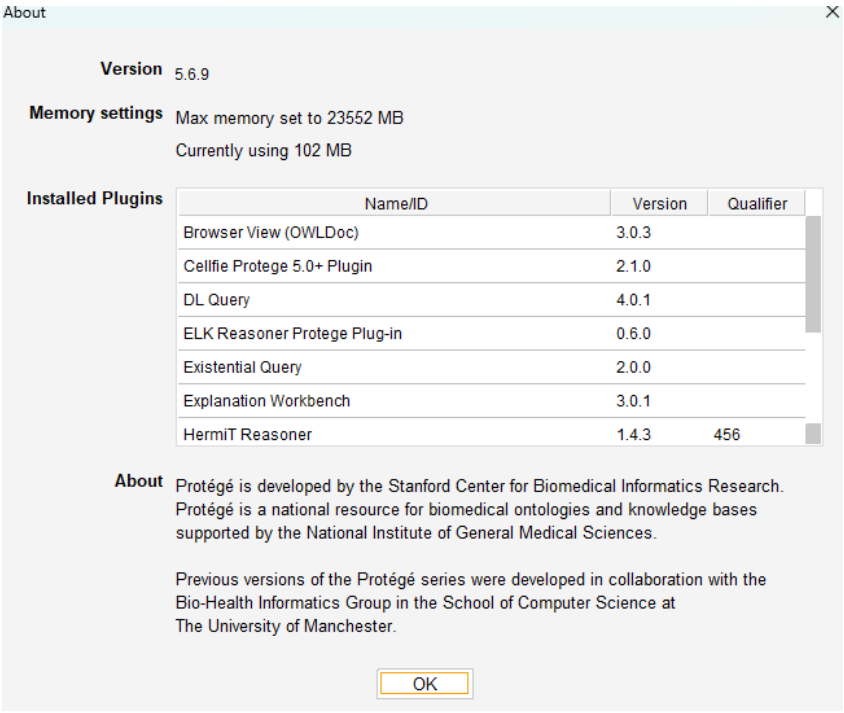


Рисунок 1 — Версия Protégé и установленные модули (Help → About)

3 ОТКРЫТИЕ ОНТОЛОГИИ

Пункты ТЗ: 4.5.3, 4.6.3. Испытания T02, T10.

1) Распакуйте архив Protégé в каталог с коротким путём без кириллицы, например `C:\Tools\Protege-5.6.9`, и запустите `Protege.exe`.

2) Выполните команду File → Open и выберите файл `ontology/videocard.owl`.

3) На вкладке Active ontology проверьте заголовок онтологии (рисунок 2): Ontology IRI — https://example.org/ontologies/videocard, Ontology Version IRI — https://example.org/ontologies/videocard/1.0.0.

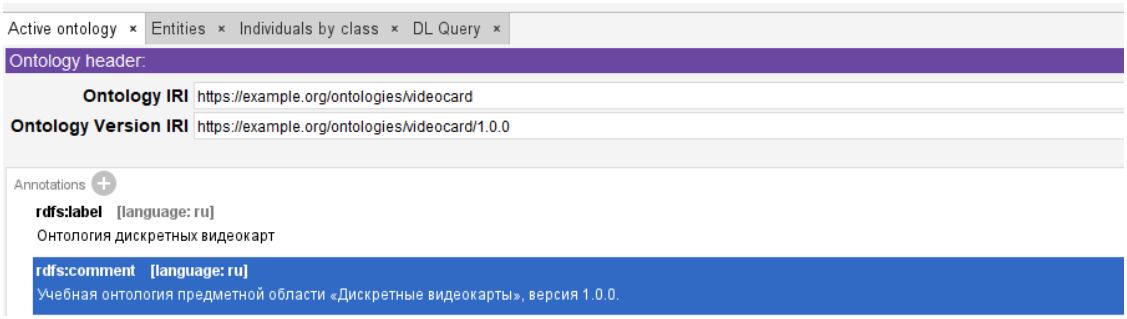


Рисунок 2 — Заголовок онтологии: IRI, version IRI, аннотации

4) В панели Ontology metrics той же вкладки сверьте метрики (рисунок 3): Axiom — 472, Logical axiom count — 253, Class count — 16 (15 пользовательских классов и owl:Thing), Object property count — 10, Data property count — 7, Individual count — 41.

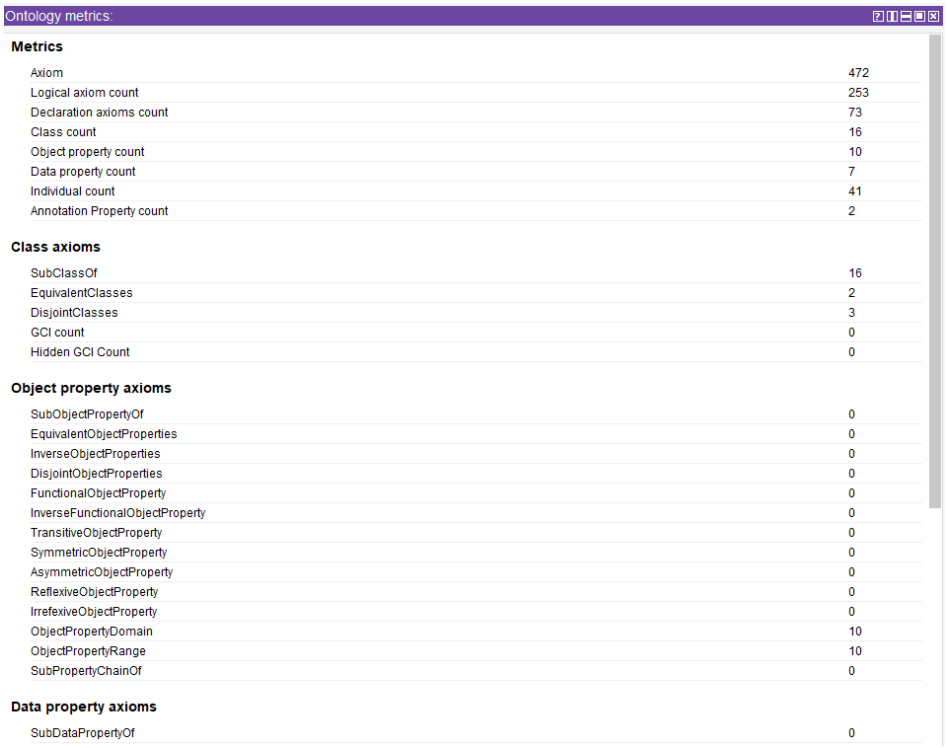


Рисунок 3 — Метрики онтологии videocard.owl

5) На вкладке Ontology imports убедитесь, что списки Direct Imports и Indirect Imports пусты (рисунок 4).

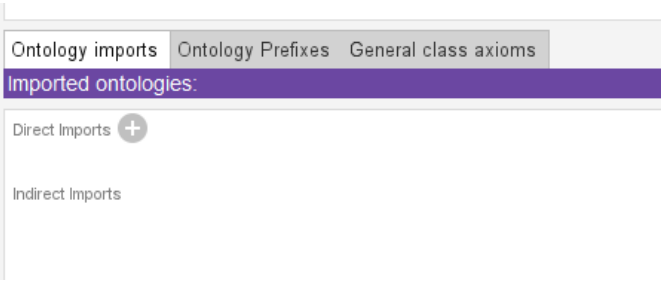


Рисунок 4 — Вкладка Ontology imports: внешних импортов нет

6) Выберите команду View → Render by entity IRI short name (Id). В этом режиме Protégé показывает технические идентификаторы, и запросы раздела 6 вводятся так, как они записаны в таблице.

Ожидаемый результат: файл открывается без сообщений об ошибках и недоступных импортах, IRI и метрики совпадают с указанными.

4 ЗАПУСК REASONER

Пункты ТЗ: 4.4.3, 4.4.6. Испытания T04, T05.

1) В меню Reasoner выберите HermiT 1.4.3.456.

2) Выполните команду Reasoner → Start reasoner (Ctrl+R). Классификация занимает несколько секунд.

3) Убедитесь, что в правом нижнем углу окна отображается надпись «Reasoner active», а флажок Show Inferences отмечен (рисунок 5). Окно «Inconsistent ontology» появляться не должно.

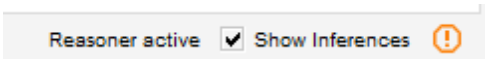


Рисунок 5 — Строка состояния после запуска HermiT

4) На вкладке Entities → Classes переключите иерархию с Asserted на Inferred. Под owl:Nothing не должно быть пользовательских классов: так Protégé показывает невыполнимые классы.

5) На вкладке Individuals by class выберите класс VideoCardModel, затем индивид U02. В панели Description в разделе Types рядом с записанным типом VideoCardModel появятся выведенные типы CardWith8GBMemory и CardWithActiveCooling на жёлтом фоне (рисунок 6).

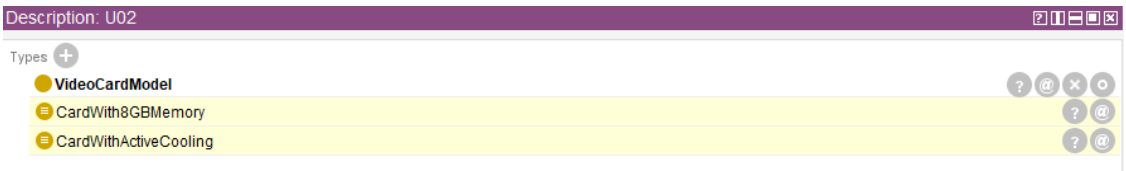


Рисунок 6 — Типы модели U02: выведенные классы выделены жёлтым

Жёлтый фон означает, что тип получен выводом, а не записан вручную. Кнопка «?» справа от выведенного типа открывает объяснение — список аксиом, из которых получен вывод.

Ожидаемый результат: онтология согласована, невыполнимых классов нет, принадлежность моделей определяемым классам соответствует таблице 3.

Таблица 3 — Ожидаемая принадлежность определяемым классам

Модель	**CardWith8GBMemory**	**CardWithActiveCooling**
U01	—	—
U02	да	да
U03	да	да
U04	да	да
U05	—	да
U06	да	—
U07	да	да
U08	—	—

5 ПРОВЕРКА ПОЛНОТЫ ЗАПОЛНЕНИЯ

Пункты ТЗ: 4.2.5, 4.4.4, 4.4.5. Испытание Т03.

Reasoner не обнаруживает пропуск обязательной связи: в предположении открытого мира отсутствующее значение считается неизвестным (п. 4.4.7 ТЗ). Поэтому полнота проверяется вручную по карточке каждой модели.

1) На вкладке Individuals by class выберите класс VideoCardModel.

2) Поочерёдно выберите индивиды U01–U08 и просмотрите панель Property assertions.

3) Отметьте в карточке полноты наличие связей по списку ниже.

Карточка полноты одной модели:

– `usesGPU` — ровно одно значение;

– `hasMemory` — ровно одно значение;

– `hasCooling` — ровно одно значение;

– `hasInterface` — ровно одно значение;

– `producedByManufacturer` — ровно одно значение;

– `hasVideoOutput` — не менее одного значения;

– `name` — задано.

Ожидаемый результат: у всех восьми моделей заданы все связи списка. Явные значения моделей приведены в таблице 10 документа «Описание онтологии».

6 ВЫПОЛНЕНИЕ КОНТРОЛЬНЫХ ЗАПРОСОВ

Пункты ТЗ: 4.3, 4.4.3. Испытание T05.

1) Запустите reasoner, как описано в разделе 4.

2) Откройте вкладку DL Query. Если вкладки нет, выберите Window → Tabs → DL Query.

3) В группе Query for отметьте флажок Instances.

4) Введите выражение из таблицы 4 в поле Query (class expression) и нажмите Execute.

5) Сравните список Instances в поле Query results с эталонным набором.

Таблица 4 — Контрольные запросы и ожидаемые результаты

№	Вопрос	Выражение DL Query	Ожидаемый результат
1	Модели производителя P1	`VideoCardModel and producedByManufacturer value P1`	U01, U02, U05, U08
2	Модели с GPU G2	`VideoCardModel and usesGPU value GPU_G2`	U02, U05
3	Память не менее 8 ГиБ	`CardWith8GBMemory`	U02, U03, U04, U06, U07
4	Тип памяти GDDR6	`VideoCardModel and hasMemory some (hasMemoryType value GDDR6)`	U02, U03, U04, U05, U06
5	Активное охлаждение	`CardWithActiveCooling`	U02, U03, U04, U05, U07
6	Видеовыход DisplayPort	`VideoCardModel and hasVideoOutput value DisplayPort`	U02, U03, U04, U07
7	Интерфейс PCIe 3.0 x16	`VideoCardModel and hasInterface value PCIe_3_0_x16`	U01, U05, U06, U08
8	Память не менее 8 ГиБ и активное охлаждение	`CardWith8GBMemory and CardWithActiveCooling`	U02, U03, U04, U07

Результат запроса 8 показан на рисунке 7.

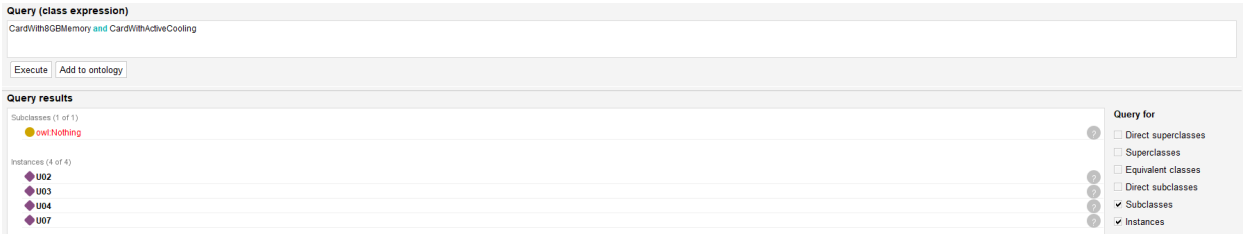


Рисунок 7 — Результат запроса 8 во вкладке DL Query

Пояснения к запросам:

– выражения записаны в синтаксисе Manchester; имена чувствительны к регистру: `GPU_G2`, а не `gpu_g2`;

– ключевое слово `value` указывает конкретный индивид, `some` — существование связи с объектом класса;

– если в разделе Subclasses результата выведен только owl:Nothing, это нормально: пользовательских подклассов у выражения нет. Ответом на вопрос служит раздел Instances;

– если включён режим отображения по меткам (View → Render by label), в запросе используются русские метки в одинарных кавычках, например `'Модель видеокарты'`. Для выражений таблицы 4 включите режим Render by entity IRI short name (Id).

Ожидаемый результат: все восемь наборов совпадают с эталоном; модели из столбца «Отр. пример» таблицы контрольных вопросов (документ «Описание онтологии», таблица 15) в ответы не попадают.

7 ПРОВЕРКА ОБНАРУЖЕНИЯ ОШИБОК

Пункты ТЗ: 4.2.3, 4.4.2, 4.4.4, 4.4.6, 4.4.7. Испытания T06–T09.

Отрицательные проверки показывают, что reasoner находит нарушения аксиом. Они выполняются только в рабочей копии; финальный файл не изменяется.

7.1 Подготовка рабочей копии

1) Скопируйте `ontology/videocard.owl` в отдельный каталог, например `tests/copy_test/`;

2) Закройте Protégé и запустите его заново: онтологию с тем же IRI, что уже загружена, Protégé в текущем окне не заменяет;

3) Откройте рабочую копию командой File → Open и запустите reasoner.

7.2 Тестовые изменения

Внесите одно изменение из таблицы 5, выполните Reasoner → Synchronize reasoner, зафиксируйте результат, затем отмените изменение (Edit → Undo, Ctrl+Z) и снова синхронизируйте reasoner.

Таблица 5 — Тестовые изменения и ожидаемая реакция

Испытание	Изменение в рабочей копии	Ожидаемая реакция
T06	Модели U01 добавить `hasCooling Cooling_Active_2fan` (у неё уже есть `Cooling_Passive`)	Окно «Inconsistent ontology»: U01 одновременно с активным и пассивным охлаждением
T07	Модели U01 добавить `usesGPU GPU_G2` (у неё уже есть `GPU_G1`)	Окно «Inconsistent ontology»: две различные GPU при ограничении exactly 1
T08	Индивиду `Mem_4GiB_GDDR5` добавить `volumeGiB` = −4 с типом xsd:integer	Окно «Inconsistent ontology»: значение вне диапазона positiveInteger
T09	У модели U02 удалить связь `hasMemory`	Сообщения нет. U02 перестаёт выводиться в CardWith8GBMemory; пропуск виден только в карточке полноты

Сообщение о несогласованности показано на рисунке 8. Кнопка Explain открывает список аксиом, которые вместе дают противоречие.

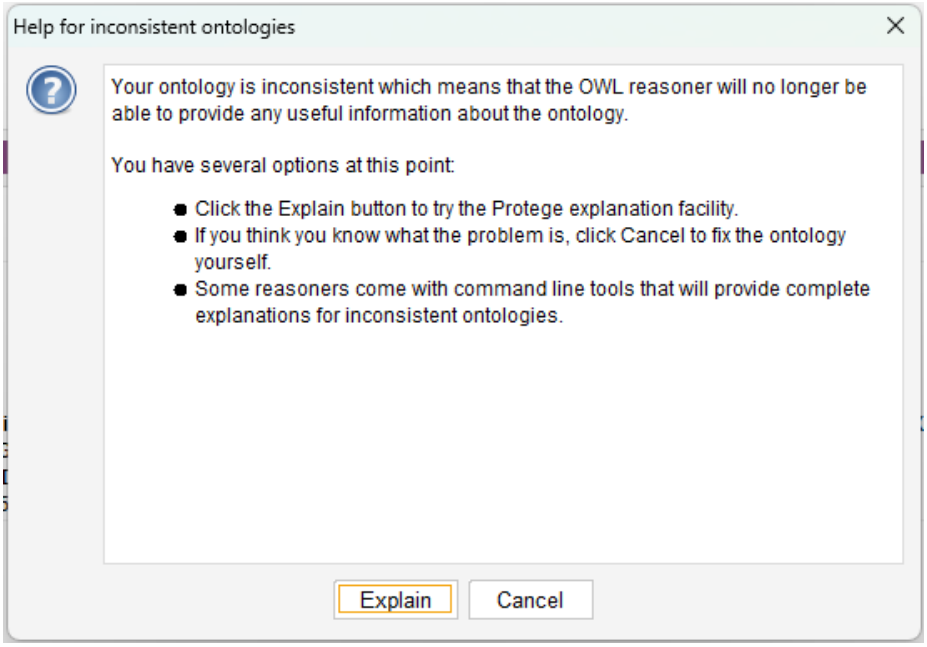


Рисунок 8 — Сообщение о несогласованности рабочей копии (испытание T06)

После каждой проверки отмените изменение и убедитесь, что в строке состояния снова «Reasoner active». Закройте рабочую копию без сохранения. Не сохраняйте её поверх `ontology/videocard.owl`.

Вместо ручного изменения можно открыть готовую копию из каталога `tests/negative/` — каждую в новом сеансе Protégé. Копии отличаются от финального файла одной правкой: `T06_active_and_passive.owl` — индивид `Cooling_Active_2fan` дополнительно отнесён к PassiveCooling; `T07_two_gpus.owl` — у модели U01 второй GPU `GPU_G2`; `T08_negative_volume.owl` — объём `Mem_4GiB_GDDR5` заменён значением −8 типа xsd:integer; `T09_missing_hasMemory.owl` — у модели U05 удалена связь `hasMemory`. Для копий T06–T08 reasoner сообщает о несогласованности, для T09 сообщения нет, а пропуск виден только в карточке U05.

8 СОХРАНЕНИЕ И ПОВТОРНОЕ ОТКРЫТИЕ

Пункты ТЗ: 4.5.1, 4.6.3. Испытание Т10.

1) Выполните команду File → Save as, выберите формат RDF/XML Syntax и сохраните онтологию в новый файл, например `videocard_resaved.owl`. Исходный файл не перезаписывайте;

2) Перезапустите Protégé и откройте сохранённый файл;

3) Сравните метрики с разделом 3, запустите reasoner и повторите запрос 8 из таблицы 4.

Ожидаемый результат: метрики совпадают, онтология согласована, запрос 8 возвращает U02, U03, U04, U07. Двоичное содержимое файлов может отличаться: для приёмки сравниваются количества сущностей и аксиом (п. 4.5.1 ТЗ).

Неизменность финального файла проверяется контрольной суммой. В PowerShell выполните:

```
Get-FileHash .\ontology\videocard.owl -Algorithm SHA256
```

Значение должно совпадать с указанным в протоколе испытаний: d8ac600a45ce642deee38be726ae5d0d81f9e47967a9e1bb07a66b5005ae3661.

9 ВОЗМОЖНЫЕ ЗАТРУДНЕНИЯ

В таблице 6 представллены возможные затруднения при работе с Protégé и их устранение.

Таблица 6 — Затруднения и их устранение

Признак	Причина	Действие
Команды меню Reasoner неактивны, в строке состояния «No Reasoner set»	Reasoner не выбран	Выбрать Reasoner → HermiT 1.4.3.456
У индивидов не видны выведенные типы на жёлтом фоне	Reasoner не запущен или снят флажок Show Inferences	Запустить reasoner (Ctrl+R), отметить Show Inferences в строке состояния
В результатах DL Query нет раздела Instances	Не отмечен флажок Instances	Отметить Instances в группе Query for
Выражение запроса подчёркнуто красным, Execute неактивна	Ошибка в имени или включён режим отображения по меткам	Сверить регистр имён с таблицей 4; выбрать View → Render by entity IRI short name (Id)
При открытии файла появляется окно «Inconsistent ontology»	Открыта рабочая копия с тестовым изменением или финальный файл изменён	Проверить путь к файлу и контрольную сумму SHA-256 (раздел 8)
При открытии второго файла Protégé сообщает, что онтология уже загружена	У файлов одинаковый IRI	Перезапустить Protégé и открыть нужный файл
Protégé не запускается или закрывается при классификации	Недостаточно памяти Java или путь к каталогу содержит кириллицу	Распаковать Protégé в каталог с латинским путём; проверить настройку памяти в Help → About

10 АВТОМАТИЧЕСКИЕ ИСПЫТАНИЯ

Испытания T01–T11 можно повторить без Protégé: скрипт открывает онтологию через OWL API 4.5.29, запускает HermiT 1.4.3.456 и сравнивает результаты с эталоном `tests/expected.json`. Нужны JDK 17 или новее (`javac` и `java` в PATH), `bash` и `curl`. Скрипт рассчитан на Linux и macOS; в Windows его запускают в WSL. Из корня комплекта выполните:

```
bash tests/run_tests.sh
```

При первом запуске скрипт загружает закреплённые jar-файлы из Maven Central (нужен доступ в Интернет), компилирует оснастку и выполняет группы T01–T11. Журналы пишутся в `tests/results/`, итог — в `tests/results/summary.json`; код возврата 0 означает, что все испытания пройдены. Последний прогон 06.10.2026: все 11 групп пройдены.

Онтология и тестовые копии собираются генератором: `bash ontology/src/build.sh` воспроизводит `ontology/videocard.owl` побайтно (контрольная сумма совпадает), `bash ontology/src/build.sh --negatives` создаёт копии T06–T09.

ПРИЛОЖЕНИЕ А

Соответствие разделов руководства испытаниям ТЗ

Таблица 7 — Соответствие разделов руководства испытаниям

Раздел руководства	Пункты ТЗ	Испытание
3 Открытие онтологии	4.2.1–4.2.4, 4.4.1, 4.5.3, 4.6.3	T02, T10
4 Запуск reasoner	4.4.2, 4.4.3, 4.4.6	T04, T05
5 Проверка полноты заполнения	4.2.5, 4.4.4, 4.4.5	T03
6 Выполнение контрольных запросов	4.3, 4.4.3	T05
7 Проверка обнаружения ошибок	4.2.3, 4.4.2, 4.4.4–4.4.7	T06–T09
8 Сохранение и повторное открытие	4.5.1, 4.6.3	T10
10 Автоматические испытания	4.3, 4.4, 4.5.1, 4.6.3	T01–T11
1, 2 Назначение, требования к среде	4.5.2, 4.6.1–4.6.4, 5	T11